TECHNICAL DOCUMENTATION

JHB CURTAIN CLEANING

Website Build Specification Document

VERSION 2.0

Z.ai Development Team
June 2026

jhbcurtaincleaning.co.za

CONTENTS

Table of Contents

SECTION 01

Project Overview

JHB CURTAIN CLEANING is Johannesburg's premier on-site curtain and blind cleaning service, serving the greater Gauteng region with professional, non-destructive cleaning solutions. The company has built a strong reputation over more than fifteen years, becoming the go-to provider for residential and commercial clients who demand museum-grade care for their window treatments. With a 4.9-star Google rating across 500+ reviews and over 5,000 clients served, the brand represents trust, quality, and deep domain expertise.

Founded by Stephen Dunlop, a certified cleaning specialist with over fifteen years of hands-on experience, JHB Curtain Cleaning has grown from a one-man operation into a full-service enterprise covering six core service categories, six industry sectors, and six geographic areas across Johannesburg and Pretoria. Stephen's personal involvement in every major project ensures that the company's exacting standards are maintained consistently, from initial consultation to final inspection.

The website, hosted at **jhbcurtaincleaning.co.za**, is built on a modern technology stack featuring Next.js 16 with the App Router, TypeScript 5, and Tailwind CSS 4. The site comprises 19 statically generated routes: one homepage, six service landing pages, six sector landing pages, and six area landing pages. All pages are pre-rendered at build time using Static Site Generation (SSG) via `generateStaticParams`, ensuring blazing-fast load times and excellent SEO performance.

Deployment is handled through Vercel's edge network, connected to a GitHub repository for continuous deployment. The architecture leverages shadcn/ui components, Framer Motion animations, and an AI-powered chatbot built on the z-ai-web-dev-sdk. A comprehensive data layer in `site-data.ts` centralizes all content, ensuring consistency across every page and component. This specification documents every facet of the build, from technology choices to component architecture, data structures, SEO strategy, and deployment infrastructure.

SECTION 02

Technology Stack

The technology stack was selected for maximum developer productivity, runtime performance, and long-term maintainability. Every dependency serves a specific architectural purpose, and the stack is optimized for static site generation with selective client-side interactivity.

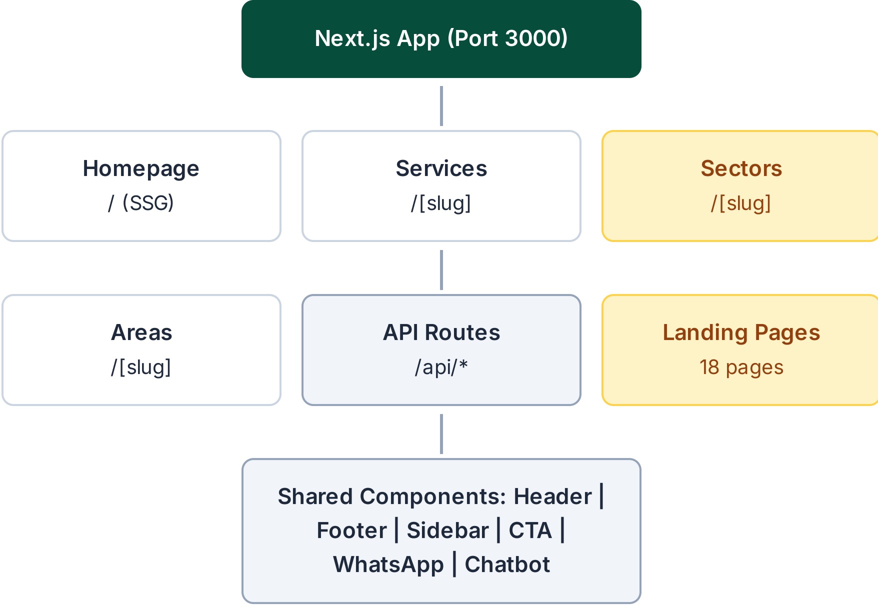
CATEGORY	TECHNOLOGY	VERSION
Framework	Next.js (App Router)	16.1.1
Language	TypeScript	5.x
Styling	Tailwind CSS	4.x
UI Library	shadcn/ui (New York)	48 components
Animations	Framer Motion	12.40.0
Forms	React Hook Form + Zod	7.60.0 / 4.0.2
ORM	Prisma	6.11.1
State	Zustand + TanStack Query	5.0.6 / 5.82.0
AI SDK	z-ai-web-dev-sdk	0.0.18
Auth	NextAuth.js	4.24.11
Runtime	Bun	Latest
Icons	Lucide React	1.21.0
Charts	Recharts	2.15.4

Next.js 16 with App Router provides the foundation for SSG, with each landing page pre-rendered at build time via `generateStaticParams`. shadcn/ui offers 48 pre-built components following the New York design variant, ensuring visual consistency. Framer Motion powers scroll-triggered animations and page transitions. The z-ai-web-dev-sdk enables the AI chatbot feature, while React Hook Form with Zod handles multi-step form validation with type-safe schemas.

SECTION 03

Site Architecture

The site follows a flat SSG architecture where all content pages are statically generated at build time. The Next.js App Router defines three dynamic route segments — services, sectors, and areas — each rendering landing pages from centralized data constants. There is no database dependency at runtime; all content is sourced from TypeScript data files.



Each dynamic route uses `generateStaticParams` to pre-render all slug variants at build time. For example, the services route iterates over the SERVICES array to generate six static HTML pages, one for each service offering. The `generateMetadata` function dynamically produces meta tags for each page, including Open Graph and Twitter Card data, ensuring comprehensive SEO coverage without any runtime computation.

The architecture is intentionally stateless at the edge. API routes handle contact form submissions, newsletter signups, and AI chatbot interactions, but none persist data in a database at this time. The Prisma ORM is configured but primarily reserved for future administrative features. The entire site, including all 19 HTML pages, is served from Vercel's CDN as static assets, guaranteeing sub-100ms Time to First Byte globally.

SECTION 04

Route Map

The site defines 19 routes across four page types and three API endpoints. Each route is statically generated from centralized data, ensuring consistency and eliminating runtime data-fetching overhead. The route patterns follow a clean URL structure optimized for both SEO and user navigation.

ROUTE PATTERN	PAGES	TYPE	DATA SOURCE
/	1	Homepage	SITE_CONFIG, SERVICES, SECTORS, AREAS, FOUNDER, FAQS
/services/[slug]	6	Service Landing	SERVICE_LANDING_DATA
/sectors/[slug]	6	Sector Landing	SECTOR_LANDING_DATA
/areas/[slug]	6	Area Landing	AREA_LANDING_DATA
/api/chat	—	API Route	z-ai-web-dev-sdk
/api/contact	—	API Route	Form handler
/api/newsletter	—	API Route	Newsletter handler

SERVICE SLUGS (6)

curtain-blind-cleaning mattress-sanitisation upholstery-cleaning master-guarding fire-proofing rug-care

SECTOR SLUGS (6)

hospitality corporate healthcare education venues residential

AREA SLUGS (6)

jhb-north jhb-east jhb-south jhb-west jhb-central pretoria-midrand

Each slug corresponds to a fully realized landing page with hero section, service details, process steps, FAQs, sidebar CTAs, and structured data. The URL pattern `/services/curtain-blind-cleaning` is both human-readable and search-engine-optimized, with the slug directly mapping to a key in the SERVICE_LANDING_DATA object for O(1) content lookup.

SECTION 05

Component Architecture

The component tree follows a strict hierarchy: RootLayout provides global structure (JSON-LD schema, fonts, metadata), while the Header and Footer wrap all pages. The homepage composes nine section components in vertical flow, and landing pages use a shared template with a two-column layout. Floating widgets (WhatsApp, AI Chatbot, Cookie Consent) exist outside the main flow.

```
RootLayout
├── JSON-LD Schema (LocalBusiness)
├── Header(sticky, glass effect)
│   ├── Logo + Brand
│   ├── NavigationMenu (Desktop)
│   │   ├── Services Dropdown (6 items)
│   │   ├── Sectors Dropdown (6 items)
│   │   └── Areas Dropdown (6 items)
│   ├── CTA Button + Phone
│   └── Sheet (Mobile Menu)
│       ├── Collapsible Sections
│       └── WhatsApp + Phone CTA
├── <main>
│   ├── Homepage Sections:
│   │   ├── HeroSection
│   │   ├── ServicesSection (6 cards)
│   │   ├── ProcessSection (4 steps)
│   │   ├── SectorsSection (6 cards)
│   │   ├── AreasSection (6 cards)
│   │   ├── AboutSection (2-col, founder)
│   │   ├── FAQSection (9 items)
│   │   ├── NewsletterStrip
│   │   ├── ContactSection (4-step form)
│   │   └── CTASection
│   └── Landing Page Template:
│       ├── LandingHero
│       ├── Guarantee Strip
│       ├── Two-Column Layout
│       │   ├── Main Content (2/3)
│       │   └── LandingSidebar (1/3, sticky)
│       ├── LandingNewsletter
│       └── LandingCTA
├── Footer (4 columns)
├── WhatsAppButton (floating)
├── CookieConsent (POPIA)
└── AIChatbot (floating widget)
```

The component architecture follows a "smart page, dumb component" philosophy. Page-level components (`page.tsx`) handle data fetching and composition, while section components are purely presentational, receiving all data via props. Client components are marked with the `"use client"` directive and are limited to interactive elements (Header, ContactSection, AIChatbot, FAQSection). The majority of the site renders as Server Components, minimizing the JavaScript bundle shipped to the client.

SECTION 06

Homepage Layout Specification

The homepage is a single-page design composed of ten vertical sections, each occupying the full viewport width with consistent horizontal padding. The sections flow in a carefully designed sequence that moves from awareness (Hero, Services) through credibility (Process, Sectors, About) to conversion (FAQ, Contact, CTA). Every section is a standalone React component.

HEADER (sticky) — Logo | Nav Dropdowns | CTA	
HERO SECTION — Tag + Heading + CTA | Trust Strip (5 badges)	
SERVICES SECTION — 6 service cards in 3×2 grid	
PROCESS SECTION — 4 numbered steps in timeline layout	
SECTORS SECTION — 6 sector cards in 3×2 grid	
AREAS SECTION — 6 area cards in 3×2 grid	
ABOUT SECTION2-col: Founder image | Bio + Stats	Stats Panel
FAQ SECTION — 9 accordion items	
NEWSLETTER STRIP — Email signup + value proposition	
CONTACT SECTION4-step form	Contact Sidebar
CTA SECTION — Full-width bronze CTA band	
FOOTER — 4 columns: Brand | Sectors | Services | Contact	

Floating Widgets (position: fixed)

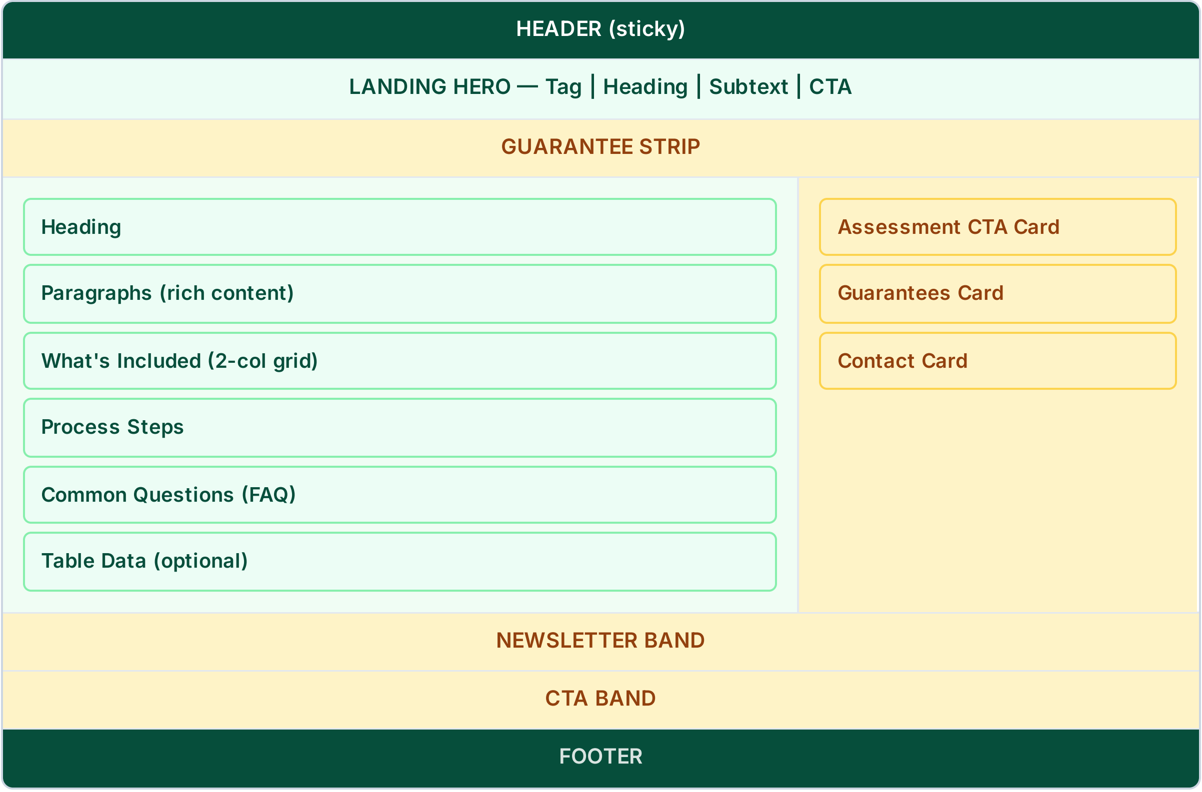
WhatsApp Button | AI Chatbot | Cookie Consent

The Hero section anchors the page with a compelling tagline, primary CTA button, and a trust strip displaying five credibility badges (Google Rating, Clients Served, Years Experience, etc.). The Services and Sectors sections use identical 3×2 card grids with hover animations, while the Process section uses a horizontal timeline with four numbered steps. The About section features a two-column layout with the founder's profile on the left and biographical content plus statistics on the right. The Contact section implements a multi-step form with smooth slide transitions between steps.

SECTION 07

Landing Page Layout Specification

All 18 landing pages (6 services, 6 sectors, 6 areas) share a common two-column template that maximizes both SEO content depth and conversion potential. The main content column (2/3 width) houses detailed service descriptions, feature grids, process steps, and FAQs, while the sidebar (1/3 width, sticky) provides persistent calls-to-action including an assessment form, guarantees card, and contact information.



The template is parameterized through data objects: SERVICE_LANDING_DATA, SECTOR_LANDING_DATA, and AREA_LANDING_DATA. Each data object contains all the content needed for a specific landing page, including hero text, features, process steps, FAQs, and sidebar configuration. The client-side components (`service-landing-client.tsx` , `sector-landing-client.tsx` , `area-landing-client.tsx`) receive this data as props and render the appropriate content blocks. This approach ensures visual consistency across all 18 landing pages while allowing each page to have unique, SEO-optimized content.

SECTION 08

Data Architecture

The data layer is centralized in `src/lib/site-data.ts` (~1,175 lines), which exports all content constants used across the site. This single-file approach ensures NAP (Name, Address, Phone) consistency, eliminates duplicate content, and makes updates trivial — change one constant, and every page reflecting that data updates automatically on the next build.

SITE_CONFIG — Central Configuration

The master configuration object containing: `name` , `shortName` , `tagline` , `domain` , `phone` , `phoneRaw` , `phoneLink` , `officePhone` , `email` , `address` , `whatsapp` , `whatsappLink` , `googleRating` , `reviewCount` , `clientsServed` , `yearsExperience` , `contactPerson` , `contactPersonNote` , `hours` (weekday/saturday), and `social` (7 platforms).

CONTENT DATA ARRAYS

DATA CONSTANT	COUNT	KEY FIELDS
SERVICES	6	id, title, shortDesc, icon, isPrimary, features[]
SECTORS	6	id, title, shortDesc, icon, features[]
AREAS	6	id, title, focus, suburbs[]
SERVICE_LANDING_DATA	6	Full page content objects (hero, features, process, faqs)
SECTOR_LANDING_DATA	6	Full page content objects
AREA_LANDING_DATA	6	Full page content objects
FAQS	9	question, answer pairs
FOUNDER	1	name, title, credentials, bio, stats

> **Supporting Data**
>
> TRUST_BADGES (5) — credibility indicators displayed in the hero section. GUARANTEES (5) — satisfaction and quality guarantees shown in landing page sidebars. PROCESS_STEPS (4) — the four-step cleaning process displayed on both the homepage and landing pages. All arrays are typed with TypeScript interfaces for compile-time safety.

The SERVICE_LANDING_DATA, SECTOR_LANDING_DATA, and AREA_LANDING_DATA objects are the most complex data structures, each containing complete page content including hero text, feature lists, process step descriptions, FAQ entries, sidebar configuration, and metadata. These objects are indexed by slug (e.g., `"curtain-blind-cleaning"`) for O(1) lookup in dynamic route handlers.

SECTION 09

Design System

The design system is built on Tailwind CSS 4 with custom theme tokens for brand colors, typography, and spacing. The palette centers on emerald green (trust, professionalism) and bronze (warmth, premium quality), with carefully chosen highlight and surface variants for interactive states and background differentiation.

COLOR PALETTE

TOKEN	VALUE	USAGE
brand-emerald	#064e3b	Primary brand, headers, CTAs
brand-bronze	#a87d43	Accent, highlights, secondary CTAs
brand-emerald-highlight	#059669	Hover states, links, interactive elements
brand-bronze-highlight	#d4a574	Light accent, subtle highlights
brand-surface-ivory	#fefce8	About section background, warm surfaces

TYPOGRAPHY

ROLE	FONT	WEIGHT	SIZE
Body	Inter	400	16px
Heading	Playfair Display	700	2xl – 5xl
Navigation	Inter	500	14px
Label	Inter	600	12px uppercase

RESPONSIVE BREAKPOINTS

BREAKPOINT	WIDTH	COLUMNS
Mobile	<640px	1
sm	640px+	1–2
md	768px+	2–3
lg	1024px+	3–4
xl	1280px+	4

The two-font system (Inter for body, Playfair Display for headings) creates a sophisticated typographic hierarchy that balances professionalism with warmth. The responsive grid system ensures content remains readable and visually balanced across all device sizes, with mobile-first design ensuring the single-column mobile experience is not an afterthought but the primary layout.

SEO & Structured Data

The site implements three layers of structured data using JSON-LD schemas, complemented by dynamic metadata generation via `generateMetadata`. This multi-layered approach ensures rich search result snippets, voice search compatibility, and knowledge graph inclusion.

1. LocalBusiness Schema (layout.tsx — site-wide)

Injected in the root layout, this schema provides Google with complete business identity information. Key fields include name, URL, email, telephone, postal address (10 2nd Ave, Florida, Roodepoort, 1710, ZA), geo coordinates (-26.1752726, 27.9233293), opening hours, price range (R800–R5500), an offer catalog with six services, and an aggregate rating (4.9 stars from 500 reviews).

```
{
  "@context": "https://schema.org",
  "@type": "LocalBusiness",
  "name": "JHB CURTAIN CLEANING",
  "url": "https://www.jhbcurtaincleaning.co.za",
  "telephone": "+27 75 011 9200",
  "address": { "@type": "PostalAddress", ... },
  "geo": { "@type": "GeoCoordinates",
    "latitude": -26.1752726, "longitude": 27.9233293 },
  "hasOfferCatalog": { 6 services },
  "aggregateRating": { "ratingValue": "4.9", "reviewCount": "500" }
}
```

2. Service Schema (per landing page)

Each service landing page includes a Service schema referencing the LocalBusiness as provider and specifying Johannesburg as the area served. This enables service-specific rich results in search, including price ranges and availability.

3. FAQPage Schema (per landing page with concerns)

Landing pages that include FAQ content also inject a FAQPage schema with Question/Answer pairs. This enables FAQ rich results in Google search, where individual questions and answers appear directly in the search snippet, dramatically increasing click-through rates.

Additional SEO Measures

generateMetadata — Each route dynamically generates meta titles, descriptions, Open Graph tags, and Twitter Card tags from data constants.

robots.txt — Configured to allow all crawlers, with sitemap reference for comprehensive indexation.

Semantic HTML — Proper use of <main>, <header>, <nav>, <section>, <article>, and <footer> elements for accessibility and SEO.

SECTION 11

API Endpoints

The site exposes three API routes, all implemented as Next.js Route Handlers with POST methods. These endpoints handle the site's interactive features: AI-powered chat, contact form submissions, and newsletter subscriptions. All endpoints are stateless and return JSON responses.

POST /api/chat — AI Chatbot

Request	{ message: string, history: Message[] }
Response	{ message: string }
SDK	z-ai-web-dev-sdk LLM completions
System Prompt	Includes all company info, services, pricing ranges, and operational details

POST /api/contact — Contact Form

Request	FormData: sector, area, name, email, phone, propertyType, windowCount, notes, popiaConsent
Response	{ success: boolean, message: string }
Validation	Zod schema with required field checks, email format, POPIA consent

POST /api/newsletter — Newsletter Signup

Request	{ email: string }
Response	{ success: boolean, message: string }
Validation	Email format validation via Zod

SECTION 12

AI Chatbot Integration

The AI chatbot is a floating widget positioned at the bottom-right of the viewport, above the WhatsApp button. It provides visitors with instant, contextual answers about services, pricing, coverage areas, and booking processes. The chatbot is powered by the z-ai-web-dev-sdk, which handles LLM completions through the `/api/chat` endpoint.

The chatbot widget opens as an expandable chat panel with a message history display. It sends user messages along with conversation history to the API endpoint, which constructs a system prompt containing all company information, service descriptions, pricing ranges, sector details, and area coverage. The AI responds contextually, directing users to the contact form or phone number when appropriate. Suggested quick actions are displayed as clickable chips for common queries like "What services do you offer?" and "How much does curtain cleaning cost?"

Error Handling & Fallback

If the AI chatbot encounters an error (network failure, API limit, etc.), it displays a fallback message with the company phone number and a prompt to call for immediate assistance. This ensures no visitor is left without a path to contact, even during service disruptions.

SECTION 13

NAP Standardization

NAP (Name, Address, Phone) consistency is critical for local SEO. The site implements the NAP Technical Doc v1.0 standard, ensuring that business identity information is byte-identical across all surfaces: JSON-LD schema, footer, contact section, and any other references. All NAP data is sourced from the SITE_CONFIG constant, eliminating the possibility of typographic drift.

NAP Format Standards

Multi-line format — Used in footer and contact page. Each element (name, address, phone) on its own line, with proper line breaks and consistent punctuation.

Single-line format — Used in social profiles and directory listings. Compressed format with pipe separators for space-constrained contexts.

JSON-LD byte-identical — The structured data schema contains the exact same text strings as the visible frontend content, ensuring Google's Knowledge Graph parser sees consistent signals.

Contact person separation — Stephen Dunlop's name and credentials are stored separately from core NAP fields, allowing them to be displayed contextually without polluting the standard NAP triplet.

Geo Coordinates

Latitude: -26.1752726, Longitude: 27.9233293. These coordinates are used in the LocalBusiness JSON-LD schema and correspond to the physical business address at 10 2nd Ave, Florida, Roodepoort, 1710, South Africa.

SECTION 14

Navigation Architecture

The navigation system provides distinct desktop and mobile experiences while maintaining information parity. On desktop, a sticky NavigationMenu offers three dropdown panels (Services, Sectors, Areas), each containing a two-column grid with icons, titles, and descriptions. On mobile, a Sheet (slide-in panel) with Collapsible sections provides the same content in a touch-friendly format.

Desktop Navigation

NavigationMenu with 3 dropdown panels. Each panel: 2-column grid with icon + title + description per item. Glass morphism effect on scroll (backdrop-blur). Active section highlighting via IntersectionObserver. Smooth scroll to #sections on homepage.

Mobile Navigation

Sheet (slide-in from right) with Collapsible accordion sections. Same content as desktop but vertically stacked. WhatsApp and Phone CTA buttons at bottom of sheet. Touch-optimized tap targets (min 44px).

The header transitions from transparent to a glass morphism effect on scroll, using `backdrop-filter: blur()` with a semi-transparent background. An IntersectionObserver tracks which homepage section is currently in view and highlights the corresponding navigation item. On the homepage, clicking any navigation item triggers a smooth scroll to the corresponding section using native `scrollIntoView({ behavior: 'smooth' })`.

SECTION 15

Contact Form System

The contact form is a four-step wizard built with React Hook Form and validated with Zod schemas. Each step presents a focused set of inputs, reducing cognitive load and increasing completion rates. Transitions between steps use Framer Motion slide animations (left for forward, right for backward).

Step 1	Step 2	Step 3
Select Sector 6 radio options	Select Area Dropdown (filtered)	Your Details Name, email, phone, etc.

Step 4: Confirmation
Review all details before submit

Step Details

Step 1 — Select Sector: Six radio button options with icons (Hospitality, Corporate, Healthcare, Education, Venues, Residential). Visual selection with active state highlighting.

Step 2 — Select Area: Dropdown menu of six Johannesburg areas, dynamically filtered based on the sector selected in Step 1.

Step 3 — Your Details: Name, email, phone, property type (dropdown), window count (number input), and notes (textarea). All fields validated with Zod schemas in real-time.

Step 4 — Confirmation: Summary review of all entered details. POPIA consent checkbox (required). Final submit button with loading state.

SECTION 16

Compliance & Security

The site is designed to comply with South Africa's Protection of Personal Information Act (POPIA), which governs how personal data must be collected, processed, and stored. POPIA compliance measures are integrated throughout the user experience, not relegated to a separate privacy page.

POPIA Compliance Measures

Cookie Consent Banner: A dismissible banner at the bottom of the viewport informs visitors about cookie usage and provides a link to the privacy policy. The banner persists until explicitly dismissed.

Privacy Policy Link: Present in the footer on every page, providing continuous access to the full privacy policy document.

POPIA Consent Checkbox: Required checkbox on the contact form (Step 4) that must be checked before submission. Labeled with clear consent language.

POPIA Compliant Badge: A green badge displayed in the footer, signaling compliance to visitors and building trust.

Data Handling Policy

All API routes are stateless — no personal data is persisted in any database. Contact form submissions and newsletter signups are processed in real-time without storage. This architecture choice eliminates data breach risk and simplifies POPIA compliance, as there is no stored personal information to protect beyond the transient request lifecycle.

SECTION 17

Deployment & Infrastructure

The site is deployed on Vercel's edge network, connected to a GitHub repository for continuous deployment. Every push to the main branch triggers an automatic build and deployment cycle. The SSG architecture means the build process generates all 19 HTML pages as static assets, which are then distributed globally via Vercel's CDN.

COMPONENT	DETAILS
Platform	Vercel (auto-deploy from GitHub)
Repository	luxrugcare-cmyk/stunning-spork
Domain	jhbcurtaincleaning.co.za (via Vercel DNS)
Build	Next.js SSG (static generation at build time)
CDN	Vercel Edge Network (global distribution)
SSL	Automatic via Vercel (HTTPS enforced)
Runtime	Bun (package manager & build tool)
Monitoring	Vercel Analytics + Web Vitals

The deployment pipeline is fully automated: code changes push to GitHub, Vercel detects the change, runs `next build` (which executes `generateStaticParams` for all routes), and deploys the resulting static assets to edge servers worldwide. SSL certificates are automatically provisioned and renewed by Vercel. The build process also generates a sitemap and robots.txt for search engine crawling. Rollback to any previous deployment is available with a single click in the Vercel dashboard.

SECTION 18

File Manifest

The following table lists all custom source files in the project, their purpose, and approximate line counts. Generated files (shadcn/ui components, Prisma client, Next.js internals) are excluded. The manifest reflects the final production codebase as deployed.

FILE	PURPOSE	LINES
src/app/layout.tsx	Root layout + JSON-LD	~230
src/app/page.tsx	Homepage composition	~41
src/app/api/chat/route.ts	AI chatbot endpoint	~92
src/app/api/contact/route.ts	Contact form handler	~60
src/app/api/newsletter/route.ts	Newsletter signup	~40
src/app/services/[slug]/page.tsx	Service SSG page	~30
src/app/services/[slug]/service-landing-client.tsx	Service landing client	~444
src/app/sectors/[slug]/page.tsx	Sector SSG page	~30
src/app/sectors/[slug]/sector-landing-client.tsx	Sector landing client	~350
src/app/areas/[slug]/page.tsx	Area SSG page	~30
src/app/areas/[slug]/area-landing-client.tsx	Area landing client	~350
src/lib/site-data.ts	Central data constants	~1,175
src/lib/data.ts	Legacy data constants	~288
src/lib/db.ts	Prisma client	~10
src/lib/utils.ts	Utility functions	~10
src/components/header.tsx	Sticky header + nav	~605
src/components/hero-section.tsx	Homepage hero	~150
src/components/services-section.tsx	Services grid	~120

src/components/process-section.tsx	Process timeline	~100
src/components/sectors-section.tsx	Sectors grid	~100
src/components/areas-section.tsx	Areas grid	~100
src/components/about-section.tsx	Founder story	~257
src/components/faq-section.tsx	FAQ accordion	~198
src/components/contact-section.tsx	Multi-step form	~844
src/components/newsletter-strip.tsx	Newsletter signup	~80
src/components/cta-section.tsx	CTA band	~60
src/components/footer.tsx	4-column footer	~186
src/components/ai-chatbot.tsx	AI chat widget	~254
src/components/whatsapp-button.tsx	WhatsApp float	~60
src/components/cookie-consent.tsx	POPIA consent	~80
src/components/landing/landing-hero.tsx	Landing hero	~80
src/components/landing/landing-sidebar.tsx	Landing sidebar	~128
src/components/landing/landing-cta.tsx	Landing CTA	~60
src/components/landing/landing-newsletter.tsx	Landing newsletter	~60

The total custom codebase comprises approximately 6,500 lines across 34 source files. The largest files are the contact section (844 lines, due to the multi-step form logic), the header (605 lines, due to desktop/mobile navigation), and the central data file (1,175 lines, containing all site content). The average component is ~150 lines, reflecting a preference for focused, single-responsibility components.

Built with precision.
Powered by AI.

JHB Curtain Cleaning | Z.ai Development